RNA-seq Nextflow Pipeline

Laurens Hahn (SaveTheWhaIes, 7307005) and Miriam Beermann (MiriamBeerm, 7452778)
Computational Workflows for Biomedical Data 2026, University of Tübingen

October 2026

1 Introduction

1.1 Pipelines

Large-scale bioinformatics data analysis, particularly in the biomedical field, requires a wide variety of tools, each with its own versions and parameters, and relies on reference data that change over time[1]. Pipelines represent data analysis as a sequence of all necessary processing steps, where the output of one step becomes the input of the next. The steps are defined once, and the entire chain can then be applied automatically to new data. Performing an analysis manually or using shell scripts makes it difficult to reproduce the results, as software updates, overwritten files, and failed steps can force the analysis to start again from scratch. Workflow managers such as Snakemake and Nextflow handle a large part of the technical work involved in an analysis: they automate individual steps, make efficient use of available resources, and perform independent tasks in parallel[1–3]. However, speed and convenience are only part of the benefit; the other part is that the results can be reproduced by others.

1.2 FAIR Principles and Reproducibility

A survey of 1,576 researchers found that more than 70 % had failed to reproduce another scientist's experiment[4]. Computational work should be easier to repeat, but missing software versions and undocumented parameters often prevent this. Workflow managers help by running tools in defined versions, together with containers and version control, and by recording what was run[1,2]. The FAIR principles ask that outputs be findable, accessible, interoperable, and reusable[5]. For a pipeline, FAIR applies not only to its outputs but to the whole process that produced them: a versioned, citable release, containerised tools[6] and tests showing stable results.

1.3 nf-core

To meet these requirements consistently, the nf-core community develops peer-reviewed Nextflow pipelines that follow shared guidelines[7]. A common template provides parameter validation, container profiles, a MultiQC report with software versions, and automated tests on GitHub[7,8]. Reusable modules wrap each tool once with a fixed container, and the nf-core tools help to create, lint, and test pipelines[7,8]. As a result, a pipeline can be started on different compute environments with a single command and a version number. One common application of such pipelines is RNA sequencing.

1.4 RNA-seq

RNA sequencing (RNA-seq) measures which genes are expressed and how strongly, and is therefore an important tool for transcriptome-wide analysis[9]. In the laboratory, RNA is extracted and converted into cDNA, which is then sequenced to produce reads[9]. Computationally, these reads typically pass through quality control and trimming, splice-aware alignment (or direct assignment to transcripts), counting per gene and comparison between samples[10]. Two expression measures are central here: raw counts,

which serve as input for differential expression analysis, for example with DESeq2[11], and transcripts per million (TPM), which are normalised for gene length and sequencing depth and allow genes within a sample to be compared[12].

1.5 Aim

The aim of this work is to develop a reproducible Nextflow workflow that follows nf-core best practices for the analysis of RNA sequencing (RNA-seq) data. To assess whether it produces reliable results, we compare its expression estimates with those of nf-core/rnaseq on two samples of a published mouse dataset[13].

2 Methods

2.1 Structure of the pipeline

The pipeline `SaveTheWhaIes/cwbd-project` was created with `nf-core pipelines create` (nf-core/tools 4.1.0) and keeps the layout of the nf-core template[7]. The most important files are:

- `main.nf`: entry point. It runs the initialisation subworkflow (parameter validation against the schema, reading the samplesheet), then the main workflow, then the completion subworkflow.
- `workflows/cwbdproject.nf`: the main workflow. It connects five local subworkflows in `subworkflows/local/`: `QC_TRIM`, `ALIGN`, `MARKDUP`, `QUANTIFY` and `ALIGNMENT_QC`, and passes all reports to MultiQC.
- `modules/nf-core/`: tool wrappers installed from the nf-core modules repository with `nf-core modules install`. Each module brings its own container and conda environment, and `modules.json` pins the commit every module was installed from. `modules/local/` holds our own two modules, `MERGE_TPM` and `MERGE_COUNTS`, whose Python scripts lie in `bin/`.
- `nextflow.config`: default parameters, profiles (`docker`, `singularity`, `conda`, `test`) and the manifest with name and version. `conf/base.config` sets CPUs and memory per process label, `conf/modules.config` sets the tool arguments (`ext.args`) and output folders per process, and `conf/test*.config` define the test data.
- `nextflow_schema.json` and `assets/schema_input.json`: JSON schemas for the parameters and the samplesheet. They are used for validation at launch and for the parameter documentation.
- `tests/`: nf-test pipeline tests with snapshot files (`*.nf.test.snap`) that store checksums of the outputs.
- `docs/usage.md`, `docs/output.md`, `README.md`, `CHANGELOG.md` and `ro-crate-metadata.json`: documentation, release history and machine-readable metadata.

2.2 Data

We used bulk RNA-seq of the nucleus accumbens from Pryce et al.[13] (GEO GSE223541, BioProject PRJNA926667), the dataset used in week 1 of the course. Two samples were analysed: SNI_Oxy_H2 (SRR23195516) and Sham_Oxy_C3 (SRR23195511), paired-end with 2×150 bp and about 21 million read pairs each. The FASTQ files were downloaded with nf-core/fetchngs 1.13.0. Both libraries are reverse stranded, as specified in the samplesheet. As the reference we used the mouse genome GRCm39 (primary assembly) and the gene annotation of Ensembl release 116[14].

2.3 Pipeline steps

The input is a samplesheet with the columns `sample`, `fastq_1`, `fastq_2` and `strandedness`. The steps and their non-default settings were:

1. **Quality control and trimming:** FastQC[15] on the raw reads, then fastp[16] with automatic adapter detection, a quality threshold of Phred 20 (`--qualified_quality_phred 20`) and a minimum read length of 36 bp after trimming (`--length_required 36`).

2. **Alignment:** HISAT2[17] with `--dta`, `--rna-strandness RF` for reverse stranded libraries, and `--no-mixed --no-discordant`, so that only concordant pairs are kept. The splice sites of the GTF are extracted with `hisat2_extract_splice_sites.py` and given to HISAT2 at alignment (`--known-splicesite-infile`). Unmapped and secondary alignments are removed, followed by sorting and indexing with SAMtools[18]. The HISAT2 index is either given (`--hisat2_index`) or built in the run. When built, splice sites and exons from the GTF are only added if the memory given by `--hisat2_build_memory` (default 200 GB) is available, otherwise a plain genome index is built. For the real data we gave a prebuilt index without splice sites and exons, as building the full index for a genome of this size needs about 200 GB of memory[19].

3. **Duplicate marking:** Picard MarkDuplicates[20] marks read pairs with identical start and end positions. Duplicates are not removed (`--REMOVE_DUPLICATES false`). SAMtools stats, flagstat and idxstats are run on the marked BAM files.

4. **Quantification:** StringTie[21] with `-e` (only annotated transcripts) and `--rf` estimates gene abundances including TPM. featureCounts from Subread[22] counts fragments on exons per gene (`-t exon -g gene_id --countReadPairs`, strand `-s 2` from the samplesheet). The local modules `MERGE_TPM` and `MERGE_COUNTS` merge the per-sample tables into one genes × samples table each.

5. **Alignment QC:** RSeQC[23] infer_experiment checks the library strandedness against the samplesheet and gives a warning on a mismatch, and read_distribution reports the share of reads on coding exons, UTRs and introns. The BED file needed by RSeQC is made from the GTF with ea-utils gtf2bed. A second featureCounts run with `-g gene_biotype` counts fragments per gene biotype, from which the share of rRNA is computed.

6. **Report:** MultiQC[24] collects the reports of FastQC, fastp, HISAT2, Picard, SAMtools, RSeQC and featureCounts together with the software versions.

2.4 Compute environment and software

All runs on real data were performed on a single Linux desktop computer at home (Intel Core i7-8700, 6 cores and 12 threads, 15 GB RAM, Ubuntu 26.04 LTS) with Nextflow 26.04.6[3], OpenJDK 25 and Docker 29.1.3, using the `docker` profile. The final run used release 1.1.0 (commit `c31c5cb`), started with `nextflow run SaveTheWhaIes/cwbd-project -r 1.1.0 -profile docker`. Every process runs in the container pinned in its module. Table 1 lists the tool versions as reported by the pipeline.

2.5 Development and testing

The repository is on GitHub (`https://github.com/SaveTheWhaIes/cwbd-project`). Each step was developed on its own feature branch and merged into `dev` through a pull request reviewed by the other team member. `main` only changes at a release. Before every pull request, `nf-core pipelines lint` had to pass without failures, and GitHub Actions ran the lint and the nf-test tests. The tests use the small yeast data from the nf-core test datasets (rnaseq branch) and cover the default run with an index built in the run, a run with a prebuilt index, a run with the full index, and a run on human chromosome 22.

Table 1: Software versions in release 1.1.0, from `pipeline_info/cwbdproject_software_mqc_versions.yml`.

Tool	Version
Nextflow	26.04.6
FastQC	0.12.1
fastp	1.3.6
HISAT2	2.2.3
SAMtools	1.24
Picard	3.5.0
StringTie	3.0.3
Subread (featureCounts)	2.1.1
RSeQC	5.0.4
MultiQC	1.35
Python (local modules)	3.14.5

2.6 Reproducibility and comparison

To test reproducibility, the pipeline was run four times from scratch on the two samples, each run with its own work directory: twice with the prebuilt index and twice with the index built in the run. Outputs of repeated runs were compared file by file with MD5 checksums, after removing lines that contain the run date, command line or work paths. The two index modes were compared by the Spearman correlation of TPM and read counts over all genes expressed in at least one of the two runs. To check the TPM values against an established workflow, they were compared in the same way to the Salmon gene TPM of the nf-core/rnaseq run on the same samples from course week 1 [25]. Run times and peak memory per process were taken from the Nextflow execution trace. To compare the pipeline with a plain shell script, we wrote a bash script that runs the same steps on the same two samples with the same containers, arguments and thread counts (taken from the task scripts of the release run). The script runs the steps one after the other and records the start and end of every step. The script is available from the authors on request.

3 Results

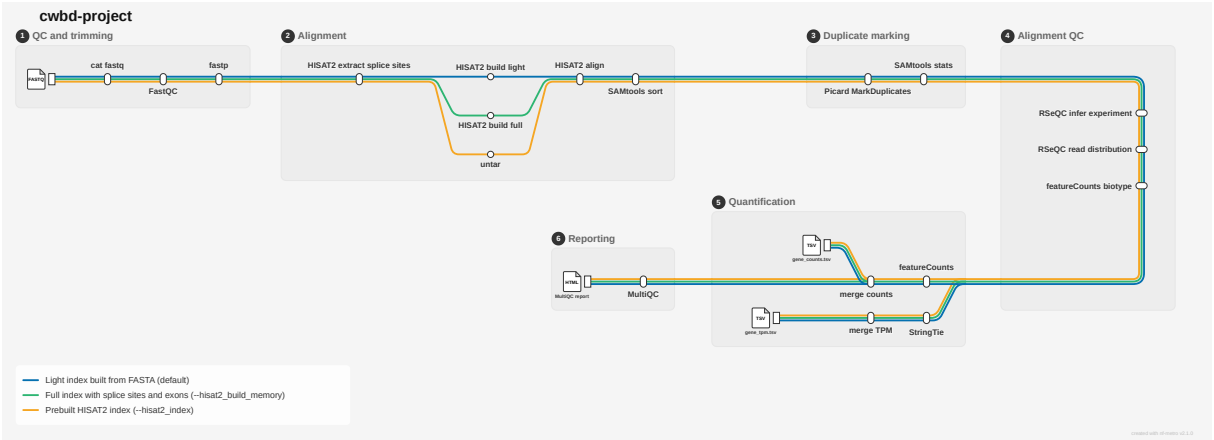


Figure 1: Overview of the pipeline as a metro map. Each line is one way to run it (index built in the run, prebuilt index, full index); stations are the processes.

3.1 Pipeline overview

The pipeline takes a samplesheet with paired-end FASTQ files and the library strandedness, a genome FASTA and a GTF annotation, and optionally a prebuilt HISAT2 index (Fig. 1). Its two main outputs are a genes × samples TPM table (`tpm/gene_tpm.tsv`) and a genes × samples read count table (`counts/gene_counts.tsv`). In addition, it writes the duplicate marked BAM files, the per sample QC reports and one MultiQC report. On the two samples, release 1.1.0 ran 33 tasks without errors.

3.2 Read quality, alignment and strandedness

Of the 20.7 and 21.5 million raw read pairs of SNI_Oxy_H2 and Sham_Oxy_C3, fastp kept 92.4 % and 95.3 %, and HISAT2 aligned 88.9 % and 86.9 % of the trimmed pairs (Fig. 2a,b). The samples differ mainly in duplication: Picard marked 69.7 % of the read pairs of SNI_Oxy_H2 as duplicates, but only 23.9 % of Sham_Oxy_C3 (Fig. 2b). Sham_Oxy_C3 also has a clearly higher share of intronic reads (Fig. 2c). RSeQC confirmed the reverse strandedness given in the samplesheet for both samples (Fig. 2d), so no warning was raised.

3.3 Gene biotypes and the 7SL RNA

About 90 % of the fragments assigned to a biotype fall on protein coding genes, and nuclear rRNA makes up only 0.01 % (Fig. 2e). The 7SL RNA (Rn7s1 and Rn7s2), however, takes 29.7 % and 17.4 % of all TPM in the StringTie table but only 0.02 % and 0.01 % of the featureCounts counts (Fig. 2f). Since the two 7SL copies are nearly identical, 78 % and 80 % of their reads multimap, which featureCounts does not count by default, and the uniquely aligned rest overlaps small same-strand genes annotated inside 7SL, which featureCounts calls ambiguous. StringTie instead distributes these reads and divides by the short gene length of 300 nt, which inflates the TPM of 7SL and lowers that of all other genes.

3.4 Comparison with Salmon

The StringTie TPM values correlate with the Salmon TPM of the nf-core/rnaseq run on the same samples with a Spearman ρ of 0.812 (SNI_Oxy_H2, 30,315 genes) and 0.783 (Sham_Oxy_C3, 35,034 genes), counting genes expressed in at least one of the two methods (Fig. 2g,h). Most genes lie slightly above the diagonal, and the largest spread is at low expression, where many genes are zero in one method but not in the other.

3.5 Reproducibility

Repeated runs from scratch gave identical results (Table 2). With the prebuilt index all 20 compared files were identical between the two runs. With the index built in the run, all files were identical except the TPM table, in which 2 of 78,348 genes differed in the sixth decimal (for example Gm15446, 15.612386 against 15.612387); the second of these runs was byte-identical to both prebuilt runs, and the 1.1.0 release run reproduced the first, so the difference is a rounding effect that flips between runs. The read count table was byte-identical in all runs. Building the index in the run and using the prebuilt index gave the same TPM and counts for every gene (Spearman $\rho = 1.000$, 100 % identical values), as both are genome indices without splice sites and exons.

Table 2: Reproducibility of four runs from scratch. Files compared after removing lines with run date, command line or work paths.

Comparison	Identical files	Differences
Prebuilt index, run 1 vs run 2	20 of 20	none
Index built in run, run 1 vs run 2	19 of 20	TPM: 2 genes in the 6th decimal
Built vs prebuilt index	TPM and counts	none ($\rho = 1.000$)

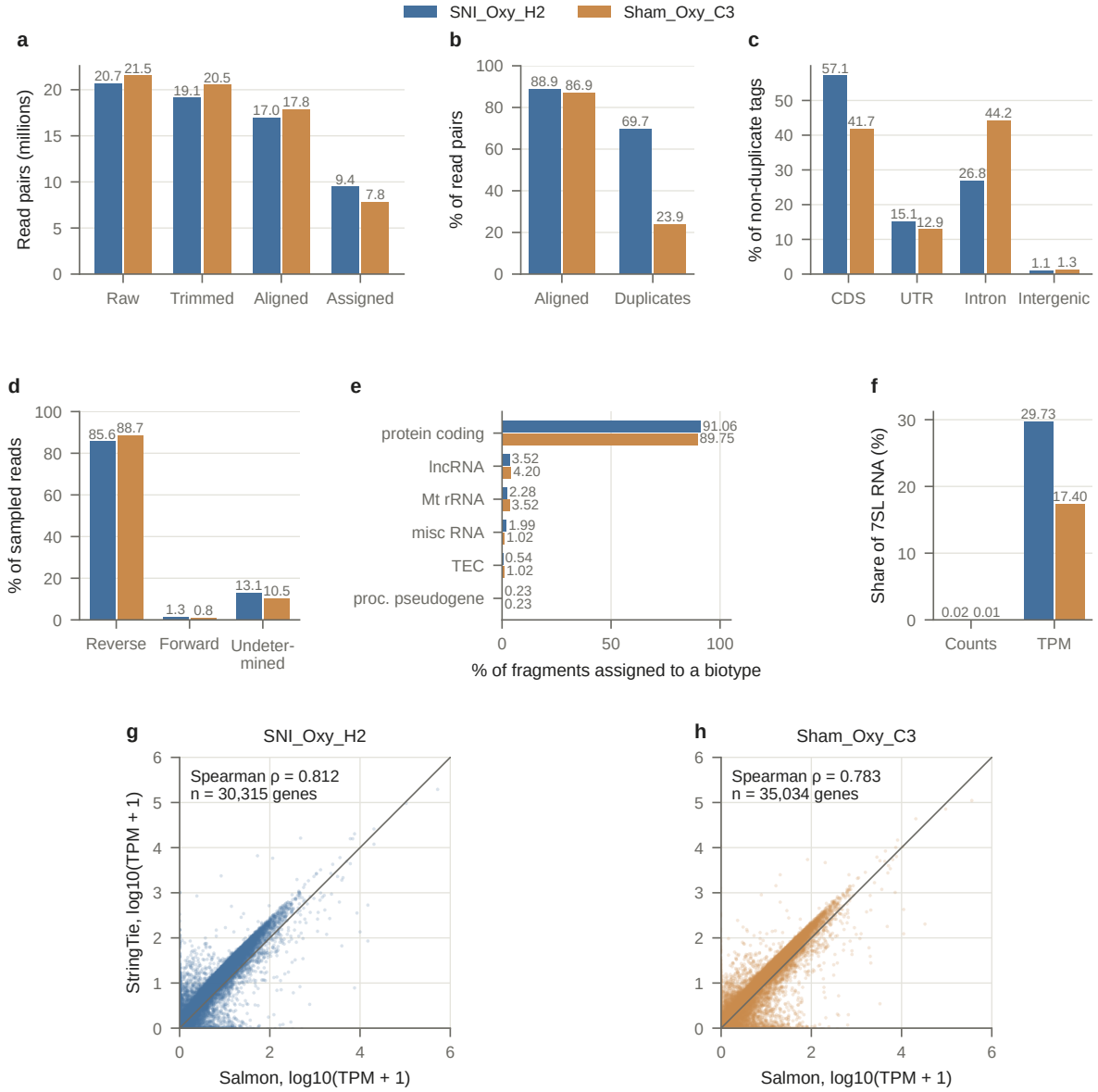


Figure 2: Quality control, gene biotypes and comparison with Salmon for the two samples. **a**, read pairs after each step (raw, after fastp, concordantly aligned, assigned to a gene by featureCounts). **b**, HISAT2 alignment rate and Picard duplicate rate. **c**, RSeQC read distribution over coding exons, UTRs, introns and intergenic regions (duplicates excluded). **d**, RSeQC strandedness check. **e**, share of fragments per gene biotype (six largest biotypes). **f**, share of 7SL RNA (Rn7s1 and Rn7s2) in the featureCounts read counts and in the StringTie TPM. **g**, **h**, gene TPM of this pipeline (StringTie) against Salmon from nf-core/rnaseq; each point is a gene expressed in at least one method, the line is $y = x$.

3.6 Run time and parallelisation

The release run took 57 minutes on the home computer, while the run times of all tasks add up to 58 minutes (Fig. 3). Apart from a few seconds, no two tasks ran at the same time, so the two samples were processed one after the other. The reason is memory: the nf-core resource labels request 12 to 36 GB per task, which our configuration caps at 14 GB, so each task reserves almost all of the 15 GB of the computer. The actual peak memory was much lower, 4.4 GB for HISAT2 and 10 GB for Picard MarkDuplicates. HISAT2 alignment (19 minutes for both samples) and duplicate marking (13 minutes) took most of the time.

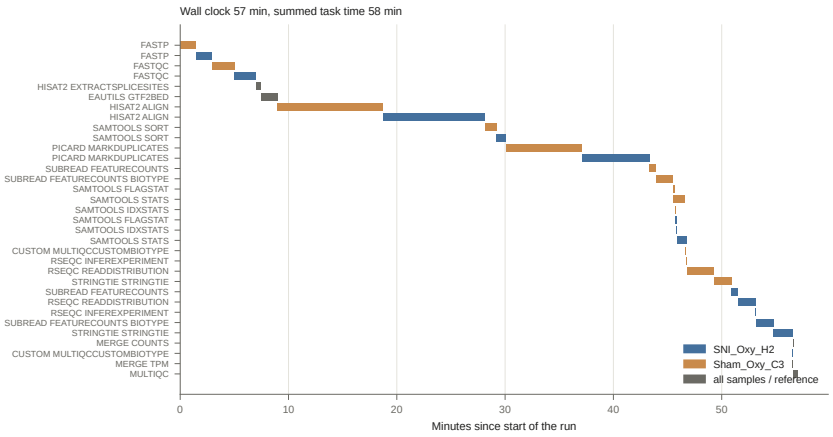


Figure 3: Timeline of all tasks of the release run, coloured by sample. Grey tasks use the reference or all samples.

The plain bash script with the same steps took 56.2 minutes, against 57.0 minutes for Nextflow (Fig. 4). The run time of each step was almost the same in both; only the conversion of the GTF to BED took longer in Nextflow (1.5 against 0.8 minutes). Both produced identical read count and TPM tables. Under these resource settings, Nextflow therefore added less than one minute of overhead but also gave no speedup, because it ran the tasks one after another just like the script.

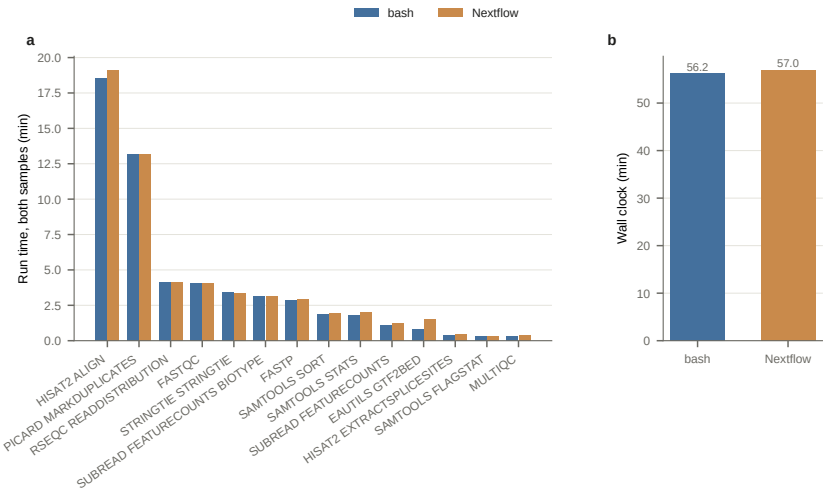


Figure 4: Run time of the pipeline in Nextflow and as a plain bash script with the same tools, containers and settings. **a**, run time per step, summed over both samples. **b**, wall clock of the whole run.

4 Discussion

Most of the reproducibility of our pipeline comes from the nf-core template, while full FAIR compliance is still missing. The template pins containers, fixes module versions in `modules.json`, validates parameters and the samplesheet at launch, provides tagged releases that can be run with `-r`, and checks snapshot checksums with nf-test on every pull request. Four runs from scratch and the release run produced identical alignments, duplicate metrics, QC reports and read counts. The only difference was found in 2 of 78,348 TPM values, which differed in the sixth decimal place, probably due to the order of floating-point summation. This has no effect on the results, but shows that byte-identical output is not guaranteed even with pinned software. Three points keep the pipeline from being fully FAIR. It is findable only via its GitHub URL, whereas a Zenodo DOI or a WorkflowHub entry would make it citable. The reference genome and annotation are recorded only as file paths, without checksums. A prebuilt HISAT2 index is opaque: we only learned that the index on our home computer lacked splice sites and exons by inspecting it, because the pipeline does not record how an index was built. Resources were a further practical issue. The nf-core label requests are written for clusters; on our 15 GB home computer each task reserved nearly all memory, so the two samples ran serially although HISAT2 peaked at only 4.4 GB. With these default requests Nextflow was as fast as a plain bash script (57.0 vs. 56.2 min). Its advantages were resume, a validated samplesheet and a software version report, which the script lacks.

Our pipeline stops at expression tables, and the 7SL RNA shows that the quantification method matters. The two 7SL copies are nearly identical, so 78 to 80 % of their reads multimap, and the uniquely aligned rest overlaps small same-strand genes annotated inside 7SL. featureCounts drops both kinds of reads, whereas StringTie distributes them. As a result, 7SL accounts for 0.02 % of the counts, but about 30 % of the TPM, which lowers the TPM of all other genes. TPM should therefore be compared between samples with care. Further gaps concern quality control. The pipeline does not have contamination screen, although human rRNA was found in 3 of 16 samples in week 1 (personal communication, tutors) and it does not remove rRNA. Duplication is reported, but PCR duplicates cannot be distinguished from natural duplicates of highly expressed genes, such as the 69.7 % duplicates of SNI_Oxy_H2. Finally, we could not test a full HISAT2 index, since it needs about 200 GB RAM for a mammalian genome[19] and STAR[26] about 32 GB, both above the 15 GB of our home computer.

4.1 Outlook

To turn this student project into a tool that others can find, cite, and run on their own data, we would first add differential expression, using the count table as input for DESeq2[11] with the 2×2 design or handing it over to nf-core/differentialabundance. Next, a contamination screen, rRNA handling, and a duplication tool such as dupRadar would close the quality-control gaps. Fractional counting of multimappers, excluding genes like 7SL, or Salmon[25], which statistically models multimappers, would address the TPM bias. Finally, a Zenodo DOI, checksums for the reference files, and a record of how indices were built would make the pipeline more FAIR and together with the steps above bring it closer to a strong RNA-seq workflow.

References

[1] Wratten, L., Wilm, A. & Göke, J. Reproducible, scalable, and shareable analysis pipelines with bioinformatics workflow managers. *Nature Methods* **18**, 1161–1168 (2021).

[2] Köster, J. & Rahmann, S. Snakemake: a scalable bioinformatics workflow engine. *Bioinformatics* **28**, 2520–2522 (2012).

[3] Di Tommaso, P. *et al.* Nextflow enables reproducible computational workflows. *Nature Biotechnology* **35**, 316–319 (2017).

[4] Baker, M. 1,500 scientists lift the lid on reproducibility. *Nature* **533**, 452–454 (2016).

[5] Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J. *et al.* The FAIR guiding principles for scientific data management and stewardship. *Scientific Data* **3**, 160018 (2016).

[6] Kurtzer, G. M., Sochat, V. & Bauer, M. W. Singularity: scientific containers for mobility of compute. *PLoS ONE* **12**, e0177459 (2017).

[7] Ewels, P. A. *et al.* The nf-core framework for community-curated bioinformatics pipelines. *Nature Biotechnology* **38**, 276–278 (2020).

[8] nf-core community. nf-core documentation. `https://nf-co.re/docs` (2026). Accessed 22 October 2026.

[9] Stark, R., Grzelak, M. & Hadfield, J. RNA sequencing: the teenage years. *Nature Reviews Genetics* **20**, 631–656 (2019).

[10] Conesa, A. *et al.* A survey of best practices for RNA-seq data analysis. *Genome Biology* **17**, 13 (2016).

[11] Love, M. I., Huber, W. & Anders, S. Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology* **15**, 550 (2014).

[12] Wagner, G. P., Kin, K. & Lynch, V. J. Measurement of mRNA abundance using RNA-seq data: RPKM measure is inconsistent among samples. *Theory in Biosciences* **131**, 281–285 (2012).

[13] Pryce, K. D. *et al.* Oxycodone withdrawal induces HDAC1/HDAC2-dependent transcriptional maladaptations in the reward pathway in a mouse model of peripheral nerve injury. *Nature Neuroscience* **26**, 1229–1244 (2023).

[14] Yates, A. D., Austine-Orimoloye, O., Azov, A. G. *et al.* Ensembl 2026. *Nucleic Acids Research* **54**, D1053–D1060 (2026).

[15] Andrews, S. FastQC: a quality control tool for high throughput sequence data. `https://www.bioinformatics.babraham.ac.uk/projects/fastqc/` (2010). Version 0.12.1.

[16] Chen, S., Zhou, Y., Chen, Y. & Gu, J. fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics* **34**, i884–i890 (2018).

[17] Kim, D., Paggi, J. M., Park, C., Bennett, C. & Salzberg, S. L. Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype. *Nature Biotechnology* **37**, 907–915 (2019).

[18] Danecek, P. *et al.* Twelve years of SAMtools and BCFtools. *GigaScience* **10**, giab008 (2021).

[19] Kim Lab. HISAT2 manual. `https://daehwankimlab.github.io/hisat2/manual/` (2026). Accessed 23 October 2026.

[20] Broad Institute. Picard toolkit. `https://broadinstitute.github.io/picard/` (2019). Version 3.5.0.

[21] Pertea, M. *et al.* StringTie enables improved reconstruction of a transcriptome from RNA-seq reads. *Nature Biotechnology* **33**, 290–295 (2015).

[22] Liao, Y., Smyth, G. K. & Shi, W. featureCounts: an efficient general purpose program for assigning sequence reads to genomic features. *Bioinformatics* **30**, 923–930 (2014).

[23] Wang, L., Wang, S. & Li, W. RSeQC: quality control of RNA-seq experiments. *Bioinformatics* **28**, 2184–2185 (2012).

[24] Ewels, P., Magnusson, M., Lundin, S. & Käller, M. MultiQC: summarize analysis results for multiple tools and samples in a single report. *Bioinformatics* **32**, 3047–3048 (2016).

[25] Patro, R., Duggal, G., Love, M. I., Irizarry, R. A. & Kingsford, C. Salmon provides fast and bias-aware quantification of transcript expression. *Nature Methods* **14**, 417–419 (2017).

[26] Dobin, A. *et al.* STAR: ultrafast universal RNA-seq aligner. *Bioinformatics* **29**, 15–21 (2013).